DATA SCIENCE & ANALYTICS · SESSION 02

Data Manipulation with Pandas

Select, filter, group, and rank — the everyday moves of real analysis

Asst.Prof.Dr.Chaiwat N. · 2026

2026 · Bachelor Program · 3 hrs · Workshop Format · Hands-on Coding

Session Agenda

01 **Recap & the Tabular Model**
Series vs DataFrame, rows, columns, the index, and dtypes

02 **Selection & Filtering**
.loc, .iloc, and the boolean masks that drive analysis

03 **Grouping & Aggregation**
groupby and the split-apply-combine pattern

04 **Sorting & Ranking**
sort_values, nlargest, and rank — within groups too

05 **Derived Columns & Apply**
New columns, vectorization, and when to reach for .apply

06 **Workshop Block**
Group, aggregate, and rank to a top-N answer — graded

01

PART 01

Recap & the Tabular Model

A quick recap of Session 1, then the mental model under everything in pandas: the Series, the DataFrame, the index, and the dtypes that decide what's possible.

Last session you loaded data and described it — today you interrogate it

Where we are

Where we were	Session 1: open a notebook, read a CSV into a DataFrame, and print the shape, head, and a few summary facts. You can load data.
Where we're going	Today: pull out exactly the rows and columns you want, group them, and rank them — the moves that turn a table into an answer.
Why it matters	Selection, grouping, and sorting answer the majority of real business questions before any modeling is involved at all.
Today's dataset	A 100-row student table — student_id, name, age, gpa, faculty, study_hours, passed_exam. Rebuilt with the same seed so everyone matches.

Source: DS Course, Session 2, 2026

Series vs DataFrame: a single column versus the whole table

The two core objects

DataFrame

The whole 2-D table — many columns, each with a name and a type. This is what read_csv hands you.

Series

A single column pulled out with df['col']. One-dimensional, still carries the index. Most operations return one.

Why it matters

Knowing which one you're holding tells you which methods are available — and whether you get a number, a column, or a table back.

PYTHON · SERIES vs FRAME

```
type(df)            # DataFrame (the table)
type(df['gpa'])     # Series    (one column)

df['gpa'].mean()    # 3.02  -> a number
df[['gpa','age']]   # -> a DataFrame

# one bracket  -> Series
# two brackets -> DataFrame
```

df['col'] is a Series; df[['col']] is a DataFrame.

Every DataFrame has the same four parts — name them and pandas stops feeling magic

The mental model

Rows	Each row is one record — here, one student. You select them by label (.loc) or by position (.iloc).
Columns	Each column is one field with a single dtype. You select them by name. A column on its own is a Series.
The index	The row labels down the left edge. Default 0,1,2,…, but it can be an ID or a date — which makes lookups and joins fast.
dtypes	The type of each column: int, float, object (text), bool, datetime. Wrong dtype, wrong result — df.info() shows them all.

Source: DS Course, Session 2, 2026

02

PART 02

Selection & Filtering

Getting exactly the rows and columns you want — label-based .loc, position-based .iloc, and the boolean masks that do most of the work in real analysis.

.loc vs .iloc: select by label, or select by position

Two ways to slice

.loc — by label

Uses index labels and column names: df.loc[0:2, ['name','gpa']]. Inclusive of the end label — a common surprise.

.iloc — by position

Uses integer positions, like a list: df.iloc[0:3, 0:4]. End-exclusive, exactly like Python slicing.

Which to use

Reach for .loc when you know the names, .iloc when you know the positions. Mixing them up is the #1 selection bug.

PYTHON · LOC & ILOC

```
# label-based: names + inclusive end
df.loc[0:2, ['name', 'gpa', 'faculty']]

# position-based: integers + exclusive
df.iloc[0:3, 0:4]

# one cell, fast
df.loc[0, 'gpa']      # 3.4
df.iloc[0, 3]         # same cell
```

.loc end is inclusive; .iloc end is exclusive.

Boolean masks: filter rows by a condition — the workhorse of every analysis

Keep the rows that match

What a mask is

A condition like df['gpa'] > 3.0 returns a column of True/False. Put it in df[...] and only the True rows survive.

Combine conditions

Use & (and) and | (or), and wrap each test in parentheses: df[(df.gpa>3) & (df.passed_exam)].

Watch out

Use & not 'and', | not 'or' — Python's words don't work element-wise. Forgetting the parentheses is the classic error.

PYTHON · BOOLEAN MASK

```
# one condition
high = df[df['gpa'] > 3.0]

# two conditions: & and |, parenthesize!
df[(df['gpa'] > 3.0) &
   (df['passed_exam'] == True)]

# membership test
df[df['faculty'].isin(['Engineering',
                       'Science'])]
```

& and | with parentheses — never 'and' / 'or'.

Selecting columns: one bracket or two decides what type you get back

Columns, precisely

One column	df['gpa'] returns a Series. Use it when you want to compute on a single field — a mean, a max, a plot.
Many columns	df[['name','gpa','faculty']] returns a DataFrame. Pass a list inside the brackets — note the double square brackets.
The [[ ]] gotcha	df['gpa'] and df[['gpa']] look almost identical but return different types. If a method complains, check which you handed it.
Chaining	Filter then select reads left to right: df[df['gpa']>3][['name','gpa']]. Get the rows, then the columns.

Source: DS Course, Session 2, 2026

03

PART 03

Grouping & Aggregation

groupby and the split-apply-combine pattern — the single most useful idea in pandas, and the one that turns 100 rows into a five-row answer.

Split-apply-combine: the one pattern behind every groupby you'll ever write

How groupby thinks

Split

Break the table into groups by a key — here, by faculty. Each faculty becomes its own little sub-table.

Apply

Run the same calculation on each group independently — a mean GPA, a count of students, a sum of hours.

Combine

Stitch the per-group results back into one tidy table, one row per group. pandas does all three in a single line.

Why it matters

Almost every 'by category' question — average X by Y, count of A per B — is a split-apply-combine in disguise.

Source: DS Course, Session 2, 2026

groupby in practice: from one metric to a full summary table

One line, then many

Single metric

df.groupby('faculty')['gpa'].mean() gives the average GPA per faculty — split, apply mean, combine.

Named aggregations

.agg() builds a whole summary table at once: count of students, average GPA, total study hours — each its own column.

Multiple keys

Group by a list — groupby(['faculty','passed_exam']) — to break results down two levels deep.

PYTHON · GROUPBY

```
# one metric per group
df.groupby('faculty')['gpa'].mean().round(2)

# a full summary table
df.groupby('faculty').agg(
    n_students=('student_id', 'count'),
    avg_gpa=('gpa', 'mean'),
    total_hours=('study_hours', 'sum'),
)
```

.agg(name=(col, func)) builds named summary columns.

pivot_table: the same split-apply-combine, laid out as a grid

When a grid beats a list

What it is	Reshapes grouped results into a 2-D grid — one variable down the rows, another across the columns, an aggregate in each cell.
index vs columns	index= sets the row groups, columns= sets the column groups, values= is what gets aggregated, aggfunc= is how.
vs groupby	Same engine underneath. Use groupby for a tidy list; use pivot_table when you want a cross-tab to read across and down.
Example	pivot_table(index='faculty', columns='passed_exam', values='gpa', aggfunc='mean') — average GPA, faculty by pass/fail.

Source: DS Course, Session 2, 2026

04

PART 04

Sorting & Ranking

sort_values, nlargest, and rank — putting the answer in order so the insight is obvious, including ranking within each group.

sort_values: order rows by one column, or by several at once

Put it in order

One key

df.sort_values('gpa', ascending=False) — highest GPA first. The fastest way to see the top and bottom of anything.

Several keys

Pass lists: sort by faculty A->Z, then GPA high->low within each. ascending takes a list to mix directions.

Missing values

na_position='last' keeps NaNs out of your top results. Sorting silently floats or sinks them otherwise.

PYTHON · SORT_VALUES

```
# single column, descending
df.sort_values('gpa', ascending=False)

# multiple columns, mixed direction
df.sort_values(
    ['faculty', 'gpa'],
    ascending=[True, False],
)

# keep NaNs out of the top
df.sort_values('gpa', na_position='last')
```

ascending takes a list to mix sort directions.

nlargest, nsmallest, and rank: the fast path to top-N and relative position

Top-N without the boilerplate

nlargest / nsmallest	df.nlargest(10, 'gpa') returns the top 10 in one call — cleaner and faster than sort_values(...).head(10).
rank	Assigns each row its position, 1 = best. df['gpa'].rank(ascending=False) turns raw values into a leaderboard column.
Tie methods	method='dense' gives 1,2,2,3; 'min' gives 1,2,2,4. Choose how ties share a rank — it changes the leaderboard.
Why prefer it	nlargest is optimized for top-N and reads as intent: 'the ten largest', not 'sort everything then chop'.

Source: DS Course, Session 2, 2026

Ranking within groups: combine groupby with rank for a per-category leaderboard

Top performers in each group

The move

groupby(key)[col].rank() ranks each row against only its own group — top student within each faculty, not overall.

Keeps its shape

rank returns one value per row, so the table keeps every row — you're adding a leaderboard column, not collapsing.

Real use

'Best in faculty', 'percentile within region', 'rank of each product in its category' are all this one pattern.

PYTHON · RANK IN GROUP

```
# rank each student within their faculty
df['rank_in_faculty'] = (
    df.groupby('faculty')['gpa']
      .rank(ascending=False, method='dense')
)

# the #1 student per faculty
df[df['rank_in_faculty'] == 1]
   [['name', 'faculty', 'gpa']]
```

groupby(...).rank() ranks within each group, not overall.

05

PART 05

Derived Columns & Apply

Create new columns from the ones you have — vectorized arithmetic first, and .apply only when row-by-row logic genuinely needs it.

Derived columns: build new fields from existing ones — vectorized

Compute across the whole column

Vectorized math

df['eff'] = df['gpa'] / df['study_hours'] runs on the entire column at once — no loop, and far faster.

Conditional columns

np.where(cond, a, b) picks per row without a loop: np.where(df.gpa>=3, 'Pass', 'Review').

Why vectorize

Operating on whole columns is both faster and clearer than looping row by row. Reach for a loop last, not first.

PYTHON · DERIVED COLUMNS

```
import numpy as np

# arithmetic across the whole column
df['study_efficiency'] = (
    df['gpa'] / df['study_hours']
)

# conditional column, no loop
df['status'] = np.where(
    df['gpa'] >= 3.0, 'Pass', 'Review')
```

Whole-column math beats looping — every time.

apply: row-by-row logic for the cases vectorization can't express

When you truly need a loop

apply a function

df.apply(fn, axis=1) runs your function on each row. Use it for multi-column logic that np.where can't capture cleanly.

The cost

apply is a Python loop under the hood — flexible but slow. Fine for 100 rows; rethink it for millions.

Rule of thumb

Try vectorized math and np.where first. Reach for apply only when the logic genuinely spans several columns with branches.

PYTHON · APPLY

```
def risk_level(row):
    if row['gpa'] < 2.5 and \
       row['study_hours'] < 3:
        return 'High Risk'
    elif row['gpa'] < 3.0:
        return 'Watch'
    return 'OK'

df['risk'] = df.apply(risk_level, axis=1)
```

apply(axis=1) = one call per row; powerful but slow.

map & replace: recode and relabel categories cleanly

Tidy up categorical values

map on a Series	Pass a dict to translate values: df['faculty'].map({'Eng':'Engineering'}). Perfect for short, fixed lookups.
replace	Swap specific values anywhere in the frame — fix typos, standardize labels, collapse categories into fewer buckets.
Binning	pd.cut turns a number into bands — gpa into 'Low/Mid/High' — so you can group and count by range, not raw value.
Why it matters	Clean, consistent categories make every later groupby and chart correct. Garbage labels quietly split your groups.

Source: DS Course, Session 2, 2026

06

PART 06

Workshop Block

Put the session together: filter, group, aggregate, and rank to produce a top-N answer — your first graded manipulation deliverable.

By the end of this block, every student has produced a ranked top-N from groupby + sort

4 Steps · 45 Minutes

Rebuild the data	Run the setup cell to recreate the 100-row student frame with the shared seed, so your numbers match the class.
Filter	Keep the rows you care about with a boolean mask — for example, only students who passed the exam.
Group & aggregate	groupby('faculty').agg(...) for the metrics you need: count, average GPA, total study hours.
Sort & take top-N	sort_values or nlargest to rank, then take the top rows. That ranked table is your graded deliverable.

Source: Workshop, Session 2, 2026

Your deliverable: a faculty leaderboard, built and ranked in one clean cell

What 'done' looks like

The task

Group by faculty, compute the metrics, and rank — a tidy table sorted best to worst, with clear column names.

Graded on

Did it run? Are the groups and numbers right? Is the output readable and sorted? Clarity beats cleverness.

Finish early?

Add rank-within-faculty, or chart the result with .plot(kind='bar'). Then try the same on a column of your choice.

PYTHON · DELIVERABLE

```
passed = df[df['passed_exam'] == True]

board = (passed
    .groupby('faculty')
    .agg(n=('student_id', 'count'),
         avg_gpa=('gpa', 'mean'),
         hours=('study_hours', 'sum'))
    .sort_values('avg_gpa', ascending=False)
    .round(2))
print(board)
```

Filter -> group -> aggregate -> sort. One readable cell.

KEY TAKEAWAY

Filter, group, sort. Three verbs answer most data questions.